Qwen

From Curation to Compression: Building High-Efficiency LLM Datasets for Resource-Constrained Training

Optimizing Data Formats and Serialization for Low-CPU Environments

The performance of large language model (LLM) training is not solely determined by the computational power of accelerators like GPUs; the host-side CPU plays a critical role, especially in resource-constrained environments [141][199]. Characterizing slowdowns in multi-GPU inference reveals that limited CPU allocations lead to significant issues such as delayed kernel launch, stalled communication, and increased tokenization latency, all of which degrade overall system performance [13][144]. Therefore, the selection of data format and serialization protocol is a foundational decision that directly impacts I/O efficiency, memory consumption, and CPU load during the data ingestion phase of training [177]. For a project with low CPU speed constraints, prioritizing data formats that minimize these overheads is paramount. The evidence strongly indicates that highly structured, columnar, or serialized binary formats offer substantial advantages over traditional text-based formats.

Apache Arrow emerges as a state-of-the-art solution for efficient data interchange, specifically designed to address the bottlenecks of data movement between processes and storage [62][129]. Its core innovation is a standardized, in-memory columnar format that allows for zero-copy reads and streaming capabilities [128][148]. This architecture drastically reduces CPU overhead because it avoids the need to deserialize and restructure data into a different format in memory [94]. When reading data stored in Apache Arrow's IPC (Inter-Process Communication) format, computations like decompression can be parallelized using a global CPU thread pool, a feature that directly benefits systems with limited processing power by maximizing throughput without creating excessive threads [192]. Furthermore, PyArrow provides robust APIs for interacting with Arrow data on disk, supporting both standard file I/O and more advanced memory-mapped file operations, which further optimizes memory usage [87]. The practical benefits are significant; for instance, streaming data from an Arrow file to a pandas DataFrame can achieve very high

data throughput, and reading raw Arrow arrays consumes minimal resident memory compared to equivalent data structures [128][146]. While there is an initial cost to serialize data into the Arrow format, this upfront investment yields profound performance gains during the training loop, where the CPU is freed from intensive parsing tasks [148]. The use of Arrow is supported by modern training frameworks and data processing libraries, making it a forward-looking choice for any serious data engineering effort [131].

Another highly effective format for minimizing I/O latency is TFRecord, a simple binary format developed by Google for TensorFlow [92]. Systems like EMLIO leverage TFRecord sharding, where data is stored in large files composed of smaller, logical records [48] [93]. This design enables efficient batch assembly, as the system can read contiguous chunks of a file rather than performing numerous small, scattered reads across many small files. This approach is particularly beneficial for distributed training systems, as it helps to saturate the network and reduce the frequency of expensive I/O calls [169]. A key advantage of TFRecord is that it avoids loading the entire dataset into CPU memory first, thereby preventing the creation of "bounce buffers" and conserving valuable RAM [94]. This makes it a robust and reliable option for large-scale machine learning applications running on hardware with constrained resources [156]. The ability to store data in a more serialized manner also facilitates various optimizations, contributing to its widespread adoption in HPC and ML contexts [92]. Storing datasets in formats like TFRecord or JSONL within a shared, distributed filesystem is a common practice for foundation model training, highlighting their suitability for scalable, efficient data access [30].

In contrast, text-based formats like JSON Lines (JSONL), while widely adopted for their human readability and simplicity, present significant inefficiencies in a low-CPU environment [47]. Each line in a JSONL file is a separate, self-contained JSON object that must be individually parsed by the CPU. This process of repeatedly invoking a parser for each line introduces considerable overhead, especially when dealing with millions or billions of examples [30]. Although JSONL is a common standard for fine-tuning datasets, its performance characteristics make it a less-than-ideal choice for the primary training set when CPU resources are scarce [212]. It can serve as a viable format for intermediate steps or for creating smaller, curated subsets for alignment or instruction tuning, but for the bulk of the training data, the performance penalty of parsing JSONL becomes prohibitive. The choice of format is not merely a matter of preference but a critical optimization that determines how efficiently the CPU can feed data to the training accelerator. Given the available options, columnar formats like Apache Arrow and Parquet, or highly serialized formats like TFRecord, represent a clear evolutionary step beyond text-based formats for achieving peak performance in data-intensive AI workloads

96 168. Their ability to streamline data access, reduce memory footprint, and offload parsing complexity to optimized backends makes them indispensable tools for building efficient datasets tailored to resource-constrained training scenarios.

Format	Primary Use Case	Key Advantages for Low-CPU Training	Key Disadvantages
Apache Arrow (IPC/Parquet)	General-purpose data interchange and storage	Zero-copy reads, streaming capabilities, reduced CPU overhead, low memory residency 128148192.	Requires an initial conversion step from source format.
TFRecord	Large-scale machine learning training	Efficient batch assembly via sharding, minimizes I/O latency, avoids bounce buffers, saves CPU memory 48 93 94.	Less suitable for interactive querying or ad-hoc analysis compared to columnar formats.
JSON Lines (JSONL)	Fine-tuning, alignment, human-readable data exchange	Human-readable, simple to generate and parse. Widely supported 30 47.	High CPU overhead due to line-by-line parsing, inefficient for large-scale training 169.

Principles of High-Quality Data Curation Under Computational Constraints

The user's directive to use "high-quality curated sources" is a critical directive, especially under the constraint of low CPU speed. In this context, "quality" extends beyond mere factual correctness to encompass structural simplicity, coherence, and low noise, all of which contribute to reducing the computational burden on both the model and the data preprocessing pipeline 85 242. A principled curation process involves a series of deliberate choices about source material, followed by an optimized and lightweight preprocessing pipeline. This ensures that the final dataset is not only rich in information relevant to the target domain but also highly efficient to ingest and process. The overarching principle for resource-constrained training is to prioritize data quality and representativeness over sheer volume, as a smaller, higher-quality dataset will almost invariably yield better results and train more efficiently than a larger, noisier one 33 35.

The selection of source data is the first and most crucial step. The ideal sources are those that are permissively licensed, extensive, and well-curated themselves. For code generation, foundational datasets include The Stack v2, a 3.1 TB collection of permissively licensed source code across 30 languages, and StarCoder2, trained on a massive 900B+ unique tokens from The Stack v2 1 2 45. These provide a broad and diverse foundation. For specialized domains, datasets like Compliance-to-Code for financial regulations 5 7, Primus for cybersecurity 29 64, and DSCodeBench for

realistic data science tasks [9] offer high-value, niche content. In the domain of conversational dialogue, high-quality, manually annotated datasets are paramount. Examples include DailyDialog, noted for its human-written and less noisy language [136], CSConv, which contains real-world customer support conversations [53], and CHARCO, which focuses on character-coherent, emotionally rich dialogues [56]. For general knowledge, large-scale ethical corpora such as Common Corpus, which comprises about two trillion tokens from open-licensed sources, provide a vast and reliable base [27] [28]. For ensuring factuality, established benchmarks like FActBench provide methodologies and testbeds for evaluating the truthfulness of generated text [107] [171].

Once high-quality sources are identified, a streamlined and efficient preprocessing pipeline is essential to minimize CPU overhead. This pipeline typically includes several stages: cleaning, normalization, deduplication, and structuring [145]. Cleaning and normalization aim to reduce noise and standardize the text. A powerful technique is Unicode normalization using standards like NFKC, which ensures consistent representation of characters [127]. Similarly, converting traditional Chinese characters to simplified variants can be done efficiently at scale using platforms like Alibaba Cloud MaxCompute [127]. Deduplication is another critical step, as duplicate data increases the memory footprint and can lead to overfitting, harming the model's ability to generalize [235] [242]. Filtering out datasets that are duplicates, have quality issues, or are not independent and identically distributed (IID) is a standard practice in large-scale data curation [243]. These bulk operations—normalization, filtering, and deduplication—should be performed on the entire corpus before it is split into smaller training samples, as this is far more computationally efficient than applying these transformations to each sample individually [145]. Tools like Data-Juicer 2.0, a data processing system with over 100 operators for text, image, and audio modalities, can automate many of these steps, enabling cloud-scale adaptive data processing [116].

Tokenization represents a foundational and often computationally expensive step in the NLP pipeline [84] [223]. The choice of tokenizer and vocabulary size significantly impacts data compression efficiency and model performance [114]. For a specific domain, training a custom tokenizer can mitigate token inflation—a phenomenon where domain-specific terms are broken down into many sub-word tokens—and improve the overall efficiency of the model [120]. For example, a tokenizer trained on Romanian text from a linguistically informed corpus showed improved performance over generic models [120]. Research into novel tokenization techniques continues, with methods like LoPace demonstrating lossless data compression through optimized Byte-Pair Encoding (BPE) and binary packing [113]. Architectural innovations like Pinned Tokenizer (PinTok) suggest dedicating

specific CPU cores to the tokenization task can reduce redundant hardware and OS overhead, further optimizing the process for low-CPU environments [255]. Ultimately, defining quality requires a pragmatic approach that balances verifiable facts, logical coherence, domain relevance, and low noise. By focusing on these principles, it is possible to construct a dataset that is not only informative but also exceptionally well-suited for efficient training on modest hardware.

Domain-Specific Strategies for Dataset Construction

While the foundational principles of data efficiency and quality apply universally, the optimal strategy for dataset construction varies significantly depending on the target domain. Each domain—general knowledge, technical documentation, conversational dialogue, and code generation—possesses unique characteristics that demand tailored approaches to sourcing, structuring, and validating data. An effective strategy must account for these differences to create a dataset that is both highly relevant and computationally manageable under low CPU speed constraints.

For the **General Knowledge** domain, the primary objective is to provide the LLM with a broad and accurate world model. The strategy should therefore emphasize breadth, verifiability, and reasoning diversity. Instead of attempting to curate a massive dataset from scratch, the most efficient approach is to start with a large, ethically-sourced corpus and then select a smaller, highly representative subset. Foundational sources include Common Corpus, which aggregates two trillion tokens from permissively licensed materials, or curated educational datasets like FineWeb-Edu [27] [28] [198]. The key challenge is to avoid model hallucination, which occurs when models generate plausible but incorrect information [31] [38]. To combat this, the curated dataset should prioritize factual accuracy. This can be achieved by using established question-answering datasets as templates for creating instruction-following examples, such as those used in the MuCPT benchmark for music-related QA [172]. Rigorous evaluation using frameworks like FActBench, which assesses factuality across multiple tasks, can guide the selection process [171]. The final dataset should consist of concise, well-defined examples covering a wide range of topics and requiring different types of reasoning, ensuring the model learns a generalized understanding of the world rather than memorizing specific passages [197].

In the **Technical Documentation** domain, the goal is to teach the model how to understand and reason about complex systems, primarily through the lens of software. The key is to structure the data to reflect the natural relationship between code and its

corresponding explanation. Developers frequently interact with documentation by pasting chunks of README files or API docs into chat interfaces to get help [161]. This behavior suggests that the most effective dataset will pair short, self-contained code snippets with their natural language descriptions. High-quality sources for this include official documentation sites like MDN Web Docs for web technologies [163] and the vast repositories of open-source projects. The curation process should involve breaking down long documents into atomic units: a single function definition, a class with its docstring, or a short code block with a comment explaining its purpose. This creates a direct prompt-completion format that is intuitive for the model to learn. The quality of this dataset hinges on the fidelity between the code and its description. Techniques for evaluating documentation-to-code traceability using LLMs can be employed to ensure that the generated explanations accurately reflect the code's behavior [162]. Adherence to standards like ISO/IEC/IEEE 26514 for user information design can also inform the creation of high-quality documentation content [60] [142].

For **Conversational Dialogue**, the focus shifts from factual accuracy and code correctness to naturalness, coherence, and engagement. The dataset should capture the dynamic and often non-linear patterns of human conversation. The starting point should be high-quality, manually annotated datasets known for their clean, human-written text, such as DailyDialog [136], or real-world interaction datasets like CSConv [53]. It is crucial to critically evaluate existing benchmarks, as some older ones may not reflect current conversational styles or safety concerns [25] [190]. A major consideration for low-CPU training is the length of the input prompt, which in dialogue systems consists of the full conversation history. To manage this, techniques for dynamic context windowing or summarization can be employed to truncate or condense the history without losing critical contextual information [211]. When generating synthetic data, it is valuable to focus on creating emotionally rich and character-coherent dialogues, as seen in the CHARCO dataset [56]. Evaluation should be multi-faceted, assessing dimensions such as engagingness, naturalness, and prosocial behavior [54] [221]. Using LLMs as automated evaluators has emerged as a scalable method for judging dialogue quality, though it should be complemented with human annotation for ground truth [21] [165].

The **Code Generation** domain is arguably the most demanding from an optimization perspective, as the ultimate measure of success is functional correctness. The strategy must therefore prioritize validation beyond syntax checking. The foundation of the dataset should be massive, permissively licensed codebases like The Stack v2 [2]. However, simply providing raw code is insufficient. The most effective approach involves execution-based validation, where generated code is automatically run against unit tests to verify its correctness [102] [122]. Benchmarks like ExeDS and XCODEEVAL are built entirely

around this principle, providing datasets of problems paired with executable tests [103][123]. Complementing this, static analysis techniques, such as using Abstract Syntax Trees (ASTs) to filter candidate code, can catch errors that would otherwise only be discovered at runtime [41]. Structuring the problem prompts clearly—such as providing a function signature and a docstring that describes the desired behavior—guides the model toward generating valid and relevant code [40]. Finally, given the legal and ethical implications, it is important to consider license compliance. Establishing a benchmark to evaluate the ability of LLMs to generate code that adheres to specific software licenses is a critical but underexplored area of research [10].

Implementation Blueprint for an Efficient Data Pipeline

Constructing a high-quality, single-domain dataset for resource-constrained LLM training requires a systematic and optimized implementation pipeline. This blueprint outlines a modular, hardware-aware process that begins with sourcing and culminates in a highly efficient, ready-to-train data format. The central theme is to perform computationally expensive operations once on large batches of data, thereby minimizing repetitive overhead during the final stages of dataset preparation. This approach directly addresses the challenge of low CPU speed by ensuring that the CPU is not wasted on redundant or inefficient tasks.

The first stage of the pipeline is **Bulk Source Processing**. This involves acquiring the raw data from the curated sources identified for the target domain. For instance, one might download the entire The Stack v2 repository for a code generation dataset or clone a GitHub repository containing technical documentation [2] [46]. The key here is to treat the source as a monolithic entity initially. All subsequent heavy lifting—such as cleaning, normalization, and deduplication—should be applied to this bulk dataset. This prevents the CPU from having to perform these costly operations on thousands or millions of individual samples later. For example, normalizing all text to the NFKC Unicode standard or removing all HTML tags from a collection of web pages should be done as a single pass over the entire corpus [127]. Similarly, employing a sophisticated deduplication algorithm to filter out near-duplicate documents or code blocks is far more efficient when applied to the whole dataset at once [235][242]. This stage leverages the fact that many cleaning and filtering operations are vectorizable or can be parallelized effectively over a large dataset.

The second stage is **Structured Curation and Annotation**. Once the raw data is cleaned, it must be transformed into a structured format appropriate for the target domain. For code generation, this means pairing problems with solutions, often by extracting function signatures and their corresponding implementations from source files. For technical documentation, it involves linking code snippets to their descriptive comments or docstrings. For conversational dialogue, it means organizing turns into coherent, multi-turn conversations. For general knowledge, this could involve transforming articles into a series of question-answer pairs or instruction-response formats. During this stage, it is also prudent to apply domain-specific filters. For example, when curating a medical dataset, one might use a keyword-based filter to extract relevant sections from Electronic Health Records (EHRs), ensuring the data is pertinent to the task [76]. This structured curation can be guided by established frameworks, such as using knowledge graphs to generate dialogue benchmarks [208][244] or leveraging SQuaRE standards for software engineering documentation [90][143].

The third and most critical stage for performance is **Efficient Serialization and Tokenization**. This is where the strategic decisions about data format and tokenization pay off. After the data is structured, it should be written out to a highly efficient binary format. As established, Apache Arrow IPC or Parquet are the superior choices due to their columnar layout and zero-copy capabilities [148][196]. Writing the entire curated dataset into a few large Arrow or Parquet files is the recommended practice. This single action dramatically reduces the I/O bottleneck during training. Concurrently, the final stage of tokenization should occur. If a custom tokenizer has been trained for the specific domain, it should be used here [120]. The tokenization process itself can be optimized; for instance, using a hybrid method that combines Zstandard compression with BPE tokenization and binary packing can yield highly compact representations of the data [113]. The output of this stage is a collection of pre-tokenized examples stored in the efficient binary format, ready for immediate consumption by the training framework.

The final stage is **Validation and Sampling**. Before the dataset is finalized, it must be validated for quality and size. For generative tasks, this involves using automated fact-checking frameworks like LEAF or FActBench to ensure the generated content is truthful and coherent [111][171]. For code, this means running the generated examples against a battery of unit tests to confirm they pass [102]. For dialogue, it may involve a round of human or LLM-based annotation to score qualities like coherence and engagement [137][165]. Once the full dataset is validated, if its size exceeds what is feasible for the target hardware, a representative subset should be sampled. This can be done using active learning techniques or by selecting examples that cover a diverse spread of topics, difficulty levels, or conversational styles, ensuring the smaller dataset retains the richness

of the original [197][253]. This final, optimized, and validated dataset is now perfectly tailored for efficient and effective training on a system with low CPU speed.

Synthesis and Final Recommendations for Dataset Development

The successful development of a high-quality, single-domain dataset for training an LLM under low CPU speed constraints necessitates a holistic strategy that prioritizes computational efficiency at every stage of the data lifecycle. The analysis confirms that the most significant performance bottlenecks in such environments are not the core model computations but the host-side CPU overhead associated with data ingestion, parsing, and I/O operations [13][141]. Consequently, a dataset designed for these conditions must be architected from the ground up with efficiency as a primary design principle, rather than being an afterthought. The optimal approach involves a synergistic combination of intelligent data format selection, a lean and hardware-aware preprocessing pipeline, a disciplined focus on data quality, and domain-specific tailoring.

The foremost recommendation is to abandon traditional text-based formats like JSONL for the primary training set and instead adopt a columnar or highly serialized binary format from the outset. **Converting all curated data into Apache Arrow IPC or Parquet format is the single most impactful action to mitigate CPU bottlenecks.** These formats eliminate the need for repetitive, CPU-intensive parsing by providing a standardized, in-memory structure that allows for zero-copy reads and efficient streaming [128][148]. This architectural choice offloads the majority of the data-handling burden from the constrained CPU to a highly optimized C++ backend, freeing up precious processing cycles for the actual training process [129]. This decision should be made early in the curation workflow, with the conversion to the efficient format being the final step before dataset delivery.

Second, the entire data preprocessing pipeline must be treated as a performance-critical component. The strategy should be to perform all heavy, bulk operations—including cleaning, normalization, and deduplication—on the entire dataset before it is segmented into training samples [145][242]. This "batch-first" approach is exponentially more efficient than applying these transformations iteratively. Furthermore, the tokenization stage warrants special attention. Investing in the creation of a domain-specific tokenizer can significantly reduce token inflation and improve data compression, leading to smaller

dataset sizes and faster processing speeds [120][215]. Techniques that combine optimized tokenization with binary packing offer a path toward even greater efficiency [113].

Third, the guiding philosophy must be "quality over quantity." For resource-constrained training, a smaller, meticulously curated dataset will consistently outperform a larger, noisier one. The effort should be concentrated on identifying and leveraging high-quality, permissively licensed source materials [2] [27] [136]. The curation process itself should be rigorous, applying checks for factual accuracy, coherence, and domain specificity, and employing established evaluation frameworks where applicable [102][171][173]. This disciplined approach ensures that every token in the dataset contributes meaningfully to the model's learning process.

Finally, while these core principles are universal, their application must be adapted to the specific demands of each domain. For **code generation**, this means implementing execution-based validation to guarantee correctness. For **conversational dialogue**, it involves managing context length and prioritizing naturalness. For **technical documentation**, it requires structuring data to explicitly link code with its natural language description. And for **general knowledge**, it translates to selecting a diverse and representative subset from a large corpus. By integrating these domain-specific tactics with the overarching principles of efficiency, it is possible to construct a dataset that is not only specialized and high-quality but is also exquisitely tailored to thrive in a low-CPU-speed training environment. This integrated methodology provides a clear and actionable path toward building effective, specialized LLMs without requiring access to state-of-the-art computational infrastructure.

Reference

1. [PDF] StarCoder2 and The Stack v2: The Next Generation - arXiv https://arxiv.org/pdf/2402.19173
2. (PDF) The Stack: 3 TB of permissively licensed source code https://www.researchgate.net/publication/365821542_The_Stack_3_TB_of_permissively_licensed_source_code
3. Data-efficient LLM Fine-tuning for Code Generation - arXiv https://arxiv.org/html/2504.12687v1

4. What's the Best and Fastest Way to Train LLM Like Gemma 3 on Big ... https://www.kaggle.com/questions-and-answers/583835
5. Enhancing Financial Compliance Checking via Code Generation https://arxiv.org/html/2505.19804v3
6. A Survey on Large Language Models for Code Generation https://dl.acm.org/doi/10.1145/3747588
7. Enhancing Financial Compliance Checking via Code Generation https://www.researchgate.net/publication/392134118_Compliance-to-Code_Enhancing_Financial_Compliance_Checking_via_Code_Generation
8. Creating Scalable Execution-based Code Generation Benchmarks https://openreview.net/forum?id=XXVRkPB1tg¬eId=Q3mNlGBXCX
9. [PDF] A Realistic Benchmark for Data Science Code Generation https://ojs.aaai.org/index.php/AAAI/article/view/40540/44501
10. A First Look at License Compliance Capability of LLMs in Code ... https://www.semanticscholar.org/paper/A-First-Look-at-License-Compliance-Capability-of-in-Xu-Gao/3774819be732ca363267a2e6d38e17a6257ba329
11. MultiFileTest: A Multi-File-Level LLM Unit Test Generation ... - arXiv https://arxiv.org/html/2502.06556v5
12. [PDF] Agent Data Science Code Generation Benchmark for Large ... https://aclanthology.org/2024.emnlp-main.748.pdf
13. Characterizing CPU-Induced Slowdowns in Multi-GPU LLM Inference https://arxiv.org/html/2603.22774v1
14. Preprocessing Techniques for LLMs: Normalization to Tokenization https://www.linkedin.com/posts/venky-thyagarajan_aitalkfortheday-preprocessing-tokenization-activity-7450648038357778432-Znko
15. Data × LLM: From Principles to Practices - arXiv https://arxiv.org/html/2505.18458v2
16. [PDF] A Rigorous Evaluation of LLM Data Generation Strategies for Low ... https://aclanthology.org/2025.emnlp-main.418.pdf
17. Achieving Peak Performance for Large Language Models https://ieeexplore.ieee.org/iel8/6287639/6514899/10589417.pdf
18. Curating Non-English Datasets for LLM Training with NVIDIA NeMo ... https://developer.nvidia.com/blog/curating-non-english-datasets-for-llm-training-with-nvidia-nemo-curator/
19. PiKa: Expert-Level Synthetic Datasets for Post-Training Alignment ... https://arxiv.org/html/2510.06670v2

20. A Survey on Recent Advances in LLM-Based Multi-turn Dialogue ... https://dl.acm.org/doi/full/10.1145/3771090
21. Large Language Models as Evaluators for Conversational ... - arXiv https://arxiv.org/html/2501.09493v2
22. [PDF] Leveraging LLMs for Dialogue Quality Measurement - ACL Anthology https://aclanthology.org/2024.naacl-industry.30.pdf
23. On the Benchmarking of LLMs for Open-Domain Dialogue Evaluation https://www.researchgate.net/publication/384211565_On_the_Benchmarking_of_LLMs_for_Open-Domain_Dialogue_Evaluation
24. Survey on evaluation methods for dialogue systems https://dl.acm.org/doi/10.1007/s10462-020-09866-x
25. On the Benchmarking of LLMs for Open-Domain Dialogue Evaluation https://arxiv.org/html/2407.03841v1
26. Evaluating LLM Chat Responses without Evaluation Dataset? - API https://community.openai.com/t/evaluating-llm-chat-responses-without-evaluation-dataset/820123
27. The Largest Collection of Ethical Data for LLM Pre-Training - arXiv https://arxiv.org/html/2506.01732v3
28. Common Corpus: The Largest Collection of Ethical Data for LLM Pre ... https://openreview.net/forum?id=0wSlFpMsGb
29. [PDF] A Pioneering Collection of Open-Source Datasets for Cybersecurity ... https://aclanthology.org/2025.emnlp-main.527.pdf
30. [PDF] Mixtera: A Data Plane for Foundation Model Training - arXiv https://arxiv.org/pdf/2502.19790
31. Hallucination to Truth: A Review of Fact-Checking and Factuality ... https://arxiv.org/html/2508.03860
32. Hallucination to truth: a review of fact-checking and factuality ... https://link.springer.com/article/10.1007/s10462-025-11454-w
33. [PDF] Structure Trumps Size: Rethinking Data Quality for LLM Reasoning https://aclanthology.org/2025.findings-emnlp.616.pdf
34. (PDF) Trustworthy Reasoning: Evaluating and Enhancing Factual ... https://www.researchgate.net/publication/394174800_Trustworthy_Reasoning_Evaluating_and_Enhancing_Factual_Accuracy_in_LLM_Intermediate_Thought_Processes
35. Curated LLM: Synergy of LLMs and Data Curation for tabular... https://openreview.net/forum?id=ynguffsGfa

36. LINS: A general medical Q&A framework for enhancing the quality ... https://www.nature.com/articles/s41467-025-64142-2

37. A comprehensive survey on integrating large language models with ... https://www.sciencedirect.com/science/article/pii/S0950705125005490

38. Evaluating and Enhancing Factual Accuracy in LLM Intermediate ... https://arxiv.org/html/2507.22940v1

39. A Benchmark Dataset for Line-Level Code Authorship Detection https://arxiv.org/html/2606.12620

40. Automatic code generation based on Abstract Syntax-based ... https://www.sciencedirect.com/science/article/pii/S0957417424026885

41. [PDF] Dynamic-Static Synergistic Selection Method for Candidate Code ... https://ojs.aaai.org/index.php/AAAI/article/view/40481/44442

42. (PDF) Rich Data Versus Quantity of Data in Code Generation AI https://www.researchgate.net/publication/392802031_Rich_Data_Versus_Quantity_of_Data_in_Code_Generation_AI_A_Paradigm_Shift_for_Healthcare

43. An evaluation study of large language models for addressing code ... https://link.springer.com/article/10.1007/s10664-026-10858-8

44. \emojidizzyStarCoder 2 and The Stack v2: The Next Generation - arXiv https://arxiv.org/html/2402.19173v1

45. The Stack: 3 TB of permissively licensed source code - OpenReview https://openreview.net/forum?id=pxpbTdUEpD

46. the-stack-dedup - OpenDataLab https://opendatalab.com/OpenDataLab/the-stack-dedup

47. Create JSONL Datasets for AI Fine-tuning with no-code or CLI https://community.openai.com/t/create-jsonl-datasets-for-ai-fine-tuning-with-no-code-or-cli/20418

48. Minimizing I/O Latency and Energy Consumption for Large-Scale AI ... https://arxiv.org/html/2508.11035v1

49. (PDF) Efficient Training of Large Language Models on Distributed ... https://www.researchgate.net/publication/382654730_Efficient_Training_of_Large_Language_Models_on_Distributed_Infrastructures_A_Survey

50. Training an Open-Source LLM on Windows (CPU-Only) - LinkedIn https://www.linkedin.com/pulse/training-opensource-llm-windows-cpuonly-manish-ojha-fdvgc

51. DialogBench: Evaluating LLMs as Human-like Dialogue Systems https://arxiv.org/html/2311.01677v2

52. [PDF] LLM-REDIAL: A Large-Scale Dataset for Conversational ... https://aclanthology.org/2024.findings-acl.529.pdf
53. [PDF] Evaluating, Synthesizing, and Enhancing for Customer Support ... https://ojs.aaai.org/index.php/AAAI/article/view/40825/44786
54. Evaluating Coherence in Dialogue Systems using Entailment https://www.researchgate.net/publication/334600233_Evaluating_Coherence_in_Dialogue_Systems_using_Entailment
55. A Method for Scoring Documents for Natural Conversation Content https://dl.acm.org/doi/full/10.1145/3706598.3714401
56. Enhancing Character-Coherent Role-Playing Dialogue with ... - MDPI https://www.mdpi.com/2078-2489/16/9/738
57. [PDF] LLM SAFETY - OpenReview https://openreview.net/pdf/49a02c4f68a8ab6fa3f6dca3f5f415dd8053a8fc.pdf
58. [PDF] Can LLM Replace Stack Overflow? A Study on Robustness and ... https://ojs.aaai.org/index.php/AAAI/article/view/30185/32103
59. A Standards-Focused Review of LLM-Based Assurance Techniques https://arxiv.org/html/2505.13766v1
60. ISO/IEC/IEEE 26514:2022(en), Systems and software engineering https://www.iso.org/obp/ui/#iso:std:iso-iec-ieee:26514:ed-1:v1:en:term:3.1.24
61. ISO/IEC 25059:2023 AI Quality Standards | PDF - Scribd https://www.scribd.com/document/916393149/1-s2-0-S1574013724000650-main
62. Data × LLM: From Principles to Practices - arXiv https://arxiv.org/html/2505.18458v3
63. [PDF] Compute Efficient Low-Memory LLM Training with Structured Sparse ... https://aclanthology.org/2024.emnlp-main.835.pdf
64. Primus: A Pioneering Collection of Open-Source Datasets for ... - arXiv https://arxiv.org/html/2502.11191v3
65. Primus: A Pioneering Collection of Open-Source Datasets for ... - arXiv https://arxiv.org/html/2502.11191v1
66. Advancing Cybersecurity LLMs Specialization via Resource-Efficient ... https://arxiv.org/html/2507.02964v1
67. From Beginner to Expert: Modeling Medical Knowledge into General ... https://arxiv.org/html/2312.01040v3
68. Thoth: Mid-Training Bridges LLMs to Time Series Understanding https://arxiv.org/html/2603.01042v1
69. Fine-Tuning and Evaluating Open-Source Large Language Models ... https://arxiv.org/html/2410.20297v1

70. Unearthing Large Scale Domain-Specific Knowledge from Public ... https://arxiv.org/html/2401.14624v4
71. Helios: A Foundational Language Model for Smart Energy ... - arXiv https://arxiv.org/html/2512.19299v1
72. LegiLM: A Fine-Tuned Legal Language Model for Data Compliance https://arxiv.org/html/2409.13721v1
73. Nemotron-CLIMB: CLustering-based Iterative Data Mixture ... - arXiv https://arxiv.org/html/2504.13161v2
74. Evaluating LLM Watermarking Methods for Medical Texts - arXiv https://arxiv.org/html/2509.07755v2
75. Evaluating LLM Watermarking Methods for Medical Texts https://www.researchgate.net/publication/395387960_Factuality_Beyond_Coherence_Evaluating_LLM_Watermarking_Methods_for_Medical_Texts
76. Ensuring Reliability of Curated Electronic Health Record–Derived ... https://pmc.ncbi.nlm.nih.gov/articles/PMC13001894/
77. A Survey on LLM-based Code Generation for Low-Resource ... - arXiv https://arxiv.org/html/2410.03981v3
78. (PDF) Low-Resource Fine-Tuning of LLMs for Domain-Specific Tasks https://www.researchgate.net/publication/396667485_Low-Resource_Fine-Tuning_of_LLMs_for_Domain-Specific_Tasks
79. On the Effectiveness of Large Language Models in Domain-Specific ... https://dl.acm.org/doi/full/10.1145/3697012
80. A Framework for Domain-Specific Dataset Creation and Adaptation ... https://www.mdpi.com/2073-431X/14/5/172
81. Domain-Specific Data Synthesis for LLMs via Minimal Sufficient... https://openreview.net/forum?id=6mpv9kG81P
82. Survey on Code Generation for Low resource and Domain Specific ... https://www.researchgate.net/publication/384700235_Survey_on_Code_Generation_for_Low_resource_and_Domain_Specific_Programming_Languages
83. Domain specific generative AI: pre-training, fine-tuning & RAG - Elastic https://www.elastic.co/search-labs/blog/domain-specific-generative-ai-pre-training-fine-tuning-rag
84. Data Preprocessing Techniques for LLMs | PDF | Learning - Scribd https://www.scribd.com/presentation/960503579/5-Data-Preprocessing-for-LLMs

85. How do I preprocess text data in a dataset for natural language ... https://milvus.io/ai-quick-reference/how-do-i-preprocess-text-data-in-a-dataset-for-natural-language-processing
86. What is the most efficient way to pass in memory data structure ... https://stackoverflow.com/questions/69033820/what-is-the-most-efficient-way-to-pass-in-memory-data-structure-tabular-like-f
87. Memory and IO Interfaces — Apache Arrow v24.0.0 https://arrow.apache.org/docs/python/memory.html
88. 1 Introduction - arXiv https://arxiv.org/html/2602.06063v1
89. Nowadays everybody wanna talk about TOON. | Milko Slavov https://www.linkedin.com/posts/milkoslavov_json-to-markdown-for-llms-performance-guide-activity-7396201422754590720-nplC
90. ISO/IEC DIS 25059(en), Software engineering https://www.iso.org/obp/ui#!iso:std:iso-iec:25059:dis:ed-2:v1:en
91. Data × LLM: From Principles to Practices - arXiv https://arxiv.org/html/2505.18458v1
92. I/O in Machine Learning Applications on HPC Systems https://dl.acm.org/doi/full/10.1145/3722215
93. [PDF] EMLIO: Minimizing I/O Latency and Energy Consumption for Large ... https://arxiv.org/pdf/2508.11035
94. [PDF] I/O in Machine Learning Applications on HPC Systems https://escholarship.org/content/qt8tc828xw/qt8tc828xw.pdf
95. Leveraging LLMs for abstracting UML and OCL representations from ... https://www.sciencedirect.com/science/article/pii/S016412122600107X
96. MDA: Persistent Learning Layer for LLMs Outperforms RAG - LinkedIn https://www.linkedin.com/posts/mert-polatt_machinelearning-llm-rag-activity-7454229051436621824-UEA2
97. Dialogue Quality and Emotion Annotations for Customer Support ... https://arxiv.org/html/2311.13910v1
98. Evaluating LLM-based Agents for Multi-turn Conversations: A Survey https://dl.acm.org/doi/10.1145/3793671
99. [PDF] Comparing LLM and human annotations of conversational safety https://aclanthology.org/2024.emnlp-main.511.pdf
100. Can LLM Replace Stack Overflow? A Study on Robustness and ... https://arxiv.org/html/2308.10335v5
101. Compiled AI: Deterministic Code Generation for LLM-Based ... - arXiv https://arxiv.org/html/2604.05150v1

102. Benchmarks and Metrics for Evaluations of Code Generation https://www.researchgate.net/publication/384348096_Benchmarks_and_Metrics_for_Evaluations_of_Code_Generation_A_Critical_Review
103. Execution-based Evaluation for Data Science Code Generation ... https://aclanthology.org/2022.dash-1.5/
104. bigcode 发布StarCoder2 代码模型（3B、15B - 火山引擎开发者社区 https://developer.volcengine.com/articles/7390577392571383847
105. PARD: Accelerating LLM Inference with Low-Cost PARallel Draft ... https://arxiv.org/html/2504.18583v1
106. Automating Research Synthesis with Domain-Specific Large ... https://dl.acm.org/doi/10.1145/3715964
107. A Framework for Evaluating Factual Consistency in Automated Text ... https://www.sciencedirect.com/science/article/pii/S0893608026006210
108. Fine-grained evaluation of a domain-specific Q&A dataset to support ... https://link.springer.com/article/10.1007/s13755-026-00458-7
109. (PDF) Knowledge Accuracy and Reducing Hallucinations in LLMs ... https://www.researchgate.net/publication/381253373_Knowledge_Accuracy_and_Reducing_Hallucinations_in_LLMs_via_Dynamic_Domain_Knowledge_Injection
110. Empirical validation of a generative AI framework for personalized ... https://www.nature.com/articles/s41598-026-42169-9
111. LEAF: Learning and Evaluation Augmented by Fact-Checking to ... https://aclanthology.org/2025.emnlp-industry.23/
112. The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs https://arxiv.org/html/2408.13296v1
113. LoPace: A Lossless Optimized Prompt Accurate Compression ... https://arxiv.org/html/2602.13266v1
114. A Family of Cross-Platform Tokenizers for Binary Analysis - arXiv https://arxiv.org/html/2511.17573v1
115. Cloud-native and Distributed Systems for Efficient and Scalable ... https://arxiv.org/html/2604.17227v1
116. Data-Juicer 2.0: Cloud-Scale Adaptive Data Processing for and with ... https://arxiv.org/html/2501.14755v2
117. Regression Language Models for Code - arXiv https://arxiv.org/html/2509.26476
118. Computer Science - arXiv https://arxiv.org/list/cs/new

119. TASE: Token Awareness and Structured Evaluation for Multilingual ... https://arxiv.org/html/2508.05468v1

120. TF3-RO-50M: Training Compact Romanian Language Models from ... https://arxiv.org/html/2601.10410v1

121. [PDF] An Auto-Constructed Benchmark for Multi-Domain Code Generation https://ojs.aaai.org/index.php/AAAI/article/view/34811/36966

122. [PDF] CREATING SCALABLE EXECUTION-BASED CODE GENERATION ... https://openreview.net/pdf?id=XXVRkPB1tg

123. [PDF] XCODEEVAL: An Execution-based Large Scale Multilingual Multitask https://aclanthology.org/2024.acl-long.367.pdf

124. Exploring different approaches to customize language models for ... https://arxiv.org/html/2603.16526v1

125. [PDF] Combining Domain and Alignment Vectors Provides Better ... https://aclanthology.org/2025.acl-short.22.pdf

126. From General to Genius: Your Strategic Guide to Domain-Specific ... https://blogs.vmware.com/cloud-foundation/2026/01/14/from-general-to-genius-your-strategic-guide-to-domain-specific-llms-for-enterprise-knowledge/

127. Platform For AI:LLM-Text Normalizer (MaxCompute) - Alibaba Cloud https://www.alibabacloud.com/help/en/pai/user-guide/text-normalizer

128. Streaming, Serialization, and IPC — Apache Arrow v24.0.0 https://arrow.apache.org/docs/python/ipc.html

129. R Interface for Apache Arrow Package | PDF | Array Data Structure https://www.scribd.com/document/584920834/Arrow

130. PyAO: PyTorch-Based Memory-Efficient LLM Training on Ethernet ... https://www.mdpi.com/1424-8220/26/7/2269

131. Streaming Real-Time Visualizations with ClickHouse, Apache Arrow ... https://clickhouse.com/blog/streaming-real-time-visualizations-clickhouse-apache-arrow-perpsective

132. Efficient Federated Learning in the Era of LLMs with Message ... https://developer.nvidia.com/blog/efficient-federated-learning-in-the-era-of-llms-with-message-quantization-and-streaming/

133. [PDF] Cost-Efficient LLM Training with Lifetime-Aware Tensor Offloading ... https://openreview.net/pdf?id=JV6ZOUb7BD

134. LLM Inference Benchmarking: Performance Tuning with TensorRT ... https://developer.nvidia.com/blog/llm-inference-benchmarking-performance-tuning-with-tensorrt-llm/

135. NaturalConv: A Chinese Dialogue Dataset Towards Multi-turn Topic ... https://arxiv.org/html/2103.02548v3

136. [PDF] DailyDialog: A Manually Labelled Multi-turn Dialogue Dataset https://aclanthology.org/I17-1099.pdf

137. Human Annotated Dialogues Dataset for Natural Conversational ... https://www.mdpi.com/2076-3417/10/3/762

138. Summary of our human annotation guidelines - ResearchGate https://www.researchgate.net/figure/Summary-of-our-human-annotation-guidelines_fig1_361060480

139. Dialogue Research-Tencent AI Lab https://ailab.tencent.com/ailab/nlp/dialogue/

140. [PDF] Making Visual Dialogue More Engaging: A New Task, Method, and ... https://ojs.aaai.org/index.php/AAAI/article/view/38650/42612

141. xLLM Technical Report - arXiv https://arxiv.org/html/2510.14686v1

142. ISO/IEC/IEEE 26514:2022(en), Systems and software engineering https://www.iso.org/obp/ui/es/#!iso:std:77451:en

143. ISO/IEC 25059:2023(en), Software engineering https://www.iso.org/obp/ui/en/#!iso:std:80655:en

144. Characterizing CPU-Induced Slowdowns in Multi-GPU LLM Inference https://arxiv.org/html/2603.22774v2

145. The Hidden Foundation of Every NLP Model: Text Preprocessing ... https://www.linkedin.com/pulse/hidden-foundation-every-nlp-model-text-preprocessing-yash-sarjekar-pobqf

146. Streaming Columnar Data with Apache Arrow - Wes McKinney https://wesmckinney.com/blog/arrow-streaming-columnar/

147. Apache Arrow File Anatomy: Buffers, Record Batches, Schemas ... https://dev.to/databro/apache-arrow-file-anatomy-buffers-record-batches-schemas-and-ipc-metadata-explained-44dp

148. Leveraging Apache Arrow for Zero-copy, Zero-serialization Cluster ... https://arxiv.org/html/2404.03030v1

149. A Comprehensive Survey of Small Language Models in the Era of ... https://dl.acm.org/doi/10.1145/3768165

150. [PDF] Knowledge-enhanced AI models for domain-specific application http://scis.scichina.com/en/2026/141101.pdf

151. Specializing LLMs to Low-Documented Domains with RAG https://link.springer.com/article/10.1007/s42979-026-04844-6

152. StarCoder 2 and The Stack v2: The Next Generation - arXiv https://arxiv.org/abs/2402.19173

153. StarCoder 2 and The Stack v2: The Next Generation - 智源社区论文 https://hub.baai.ac.cn/paper/73309d08-78b6-4acd-8fe8-a718d0bd3717

154. StarCoder 2 and The Stack v2: The Next Generation - 知乎专栏 https://zhuanlan.zhihu.com/p/32742509856

155. Federico Cassano's Post - StarCoder2 and The Stack v2 - LinkedIn https://www.linkedin.com/posts/federicocassano_starcoder2-and-the-stack-v2-activity-7169371697131765761-3kfi

156. I/O in Machine Learning Applications on HPC Systems: A 360 ... https://dl.acm.org/doi/10.1145/3722215

157. VLA Foundry: A Unified Framework for Training Vision-Language ... https://arxiv.org/html/2604.19728v1

158. Amazon Machine Learning – Artificial Intelligence - AWS https://aws.amazon.com/blogs/machine-learning/tag/amazon-machine-learning/feed/

159. [PDF] Gaperon: A Peppered English-French Generative Language Model ... https://arxiv.org/pdf/2510.25771

160. 1 Introduction - arXiv https://arxiv.org/html/2411.15100v3

161. Optimizing technical documentations for LLMs - DEV Community https://dev.to/joshtom/optimizing-technical-documentations-for-llms-4bcd

162. Evaluating the Use of LLMs for Documentation to Code Traceability https://arxiv.org/html/2506.16440v1

163. MDN Web Docs - Mozilla https://developer.mozilla.org/en-US/

164. [PDF] WORLD DEVELOPMENT REPORT 2026 https://thedocs.worldbank.org/en/doc/1e4e52502104a331fb42cba0d4afa995-0050062026/original/WDR2026-Concept-Note.pdf

165. A Multi-Turn Emotionally-Rich Spoken Dialogue Dataset - arXiv https://arxiv.org/html/2505.19978v1

166. Human Annotated Dialogues Dataset for Natural Conversational ... https://www.researchgate.net/publication/338728764_Human_Annotated_Dialogues_Dataset_for_Natural_Conversational_Agents

167. Setting up training jobs to access datasets - Amazon SageMaker AI https://docs.aws.amazon.com/sagemaker/latest/dg/model-access-training-data.html

168. Multimodal fusion with vision-language-action models for robotic ... https://www.sciencedirect.com/science/article/pii/S1566253525011248

169. The bottlenecks of distributed active inference.. | Denis O. - LinkedIn https://www.linkedin.com/posts/denis-o-b61a379a_ai-activity-7368112333262389248-Q720

170. Hallucination to Truth: A Review of Fact-Checking and Factuality ... https://arxiv.org/html/2508.03860v2

171. [PDF] FActBench: A Benchmark for Fine-grained Automatic Evaluation of ... https://aclanthology.org/2025.icnlsp-1.11.pdf

172. [PDF] MuCPT: Music-related Natural Language Model Continued Pretraining https://openreview.net/pdf?id=9gL6FYoBNU

173. Evaluating LLM Watermarking Methods for Medical Texts - arXiv https://arxiv.org/html/2509.07755v1

174. Research on the development of an automated system for ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC13108753/

175. AutoAdapt: An Automated Domain Adaptation Framework for Large ... https://arxiv.org/html/2603.08181v1

176. Build an LLM Text Processing Pipeline: Tokenization & Vocabulary ... https://www.linkedin.com/pulse/build-llm-text-processing-pipeline-tokenization-day-2-shanoj-kumar-v-5me1c

177. Before the First Token: Benchmarking Data Preprocessing in Vision ... https://dl.acm.org/doi/pdf/10.1145/3805621.3807641

178. A Survey on LLM-based Code Generation for Low-Resource and ... https://www.researchgate.net/publication/396307682_A_Survey_on_LLM-based_Code_Generation_for_Low-Resource_and_Domain-Specific_Programming_Languages

179. [PDF] Exploring different approaches to customize language models for ... https://arxiv.org/pdf/2603.16526

180. [PDF] Integrating Domain Knowledge into LLM Code Generation https://www.cs.cmu.edu/afs/cs.cmu.edu/Web/Posters/S3DProposal-SE-ManishaMukherjee25.pdf

181. [PDF] On Synthetic Data Strategies for Domain-Specific Generative Retrieval https://aclanthology.org/2025.acl-long.392.pdf

182. Rapids Introduction and Benchmark - Leadtek AI Forum https://forums.leadtek.com/en/post/6

183. Scalability Evaluation of HPC Multi-GPU Training for ECG-based LLMs https://arxiv.org/html/2503.21033v1

184. Apache Spark Streaming, Kafka and HarmonicIO: A Performance ... https://www.researchgate.net/publication/342022863_Apache_Spark_Streaming_Kafka_and_HarmonicIO_A_Performance_Benchmark_and_Architecture_Comparison_for_Enterprise_and_Scientific_Computing

185. Optimize for CPU Throughput with Apache Arrow Flight - LinkedIn https://www.linkedin.com/posts/isparshp_the-most-efficient-serialization-strategy-activity-7411514097378455552-jlcH

186. Zenodo https://zenodo.org/

187. A Survey on Recent Advances in Conversational Data Generation https://arxiv.org/html/2405.13003v2

188. A Survey on Recent Advances in Conversational Data Generation https://dl.acm.org/doi/10.1145/3795686

189. [PDF] CMT-Eval: A Novel Chinese Multi-turn Dialogue Evaluation Dataset ... https://aclanthology.org/2025.findings-emnlp.992.pdf

190. SafeDialBench: A Fine-Grained Safety Evaluation Benchmark for ... https://openreview.net/forum?id=KFjtRqVnKH

191. On the Effectiveness of Large Language Models in Domain-Specific ... https://arxiv.org/html/2312.01639v2

192. Arrow IPC — Apache Arrow v24.0.0 https://arrow.apache.org/docs/cpp/api/ipc.html

193. Apache Arrow IPC streams: SPMC concurrency - Stack Overflow https://stackoverflow.com/questions/76812485/apache-arrow-ipc-streams-spmc-concurrency

194. Serialization and IPC — Apache Arrow v8.0.0 https://arrow.apache.org/docs/8.0/python/api/ipc.html

195. An Evaluation of LLMs Inference on Popular Single-board Computers https://arxiv.org/html/2511.07425v1

196. Querying Parquet with Millisecond Latency - Apache Arrow https://arrow.apache.org/blog/2022/12/26/querying-parquet-with-millisecond-latency/

197. Is This Collection Worth My LLM's Time? Automatically Measuring ... https://arxiv.org/html/2502.13691v2

198. [PDF] Multi-Property Document Annotation for LLM Data Curation at Scale https://arxiv.org/pdf/2602.12414

199. [PDF] A Resource-Frugal LLM Serving Framework by Optimizing End-to ... https://aclanthology.org/2024.emnlp-industry.22.pdf

200. ISO/IEC/IEEE 26514:2022 Most Recent - Accuris Standards Store https://store.accuristech.com/standards/iso-iec-ieee-26514-2022?product_id=2500883&srsltid=AfmBOorwembEg6ASmK648PCbFCj2ucsrkNL8nwq5kqkhq_9nIDmgSwUm

201. Thousand-GPU Large-Scale Training and Optimization Recipe for AI ... https://arxiv.org/html/2603.11101v1

202. [PDF] Training-Free Token Reduction for MLLM Acceleration https://ojs.aaai.org/index.php/AAAI/article/view/42460/46421

203. starcoder2 - BigCode - Kaggle https://www.kaggle.com/models/bigcode-project/starcoder2/discussion/487804

204. Text-to-Pipeline: Bridging Natural Language and Data Preparation ... https://arxiv.org/html/2505.15874v2

205. Bench360—Benchmarking Local LLM inference from 360° - arXiv https://arxiv.org/html/2511.16682v1

206. LLM Inference Benchmarking: Fundamental Concepts https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/

207. How to Benchmark LLM Inference Performance: TTFT, ITL, and ... https://dev.to/wheynelau/how-to-benchmark-llm-inference-performance-ttft-itl-and-throughput-metrics-416p

208. Dialogue Benchmark Generation from Knowledge Graphs with Cost ... https://arxiv.org/html/2501.09928v1

209. [PDF] Talk2Code: A Multi-Turn Interaction Benchmark with Dual-Track ... https://ojs.aaai.org/index.php/AAAI/article/view/40730/44691

210. NTPP: Generative Speech Language Modeling for Dual-Channel ... https://openreview.net/forum?id=5Gke1dfRVA

211. Dynamic Token Pruning for Efficient Long Context LLM Inference https://machinelearning.apple.com/research/dynamic-token-pruning

212. Metadata for LLM Training - Juan Moisés de la Serna - Zenodo https://zenodo.org/records/19607129

213. 1 Introduction - arXiv https://arxiv.org/html/2606.07362

214. Metrics — NVIDIA NIM LLMs Benchmarking https://docs.nvidia.com/nim/benchmarking/llm/latest/metrics.html

215. Getting the most out of your tokenizer for pre-training and domain ... https://arxiv.org/html/2402.01035v2

216. [PDF] A Regex Minimization Benchmark: A PSPACE-Complete Challenge ... https://aclanthology.org/2026.eacl-long.375.pdf

217. Research on Code Generation Technology based on LLM Pre-training https://www.researchgate.net/publication/385922965_Research_on_Code_Generation_Technology_based_on_LLM_Pre-training

218. [PDF] CONV-TO-BENCH: EVALUATING LANGUAGE MODELS https://openreview.net/pdf?id=PmkOtKBMr0

219. [PDF] Optimizing Large Language Model Inference on Intel® Processors ... https://cdrdv2-public.intel.com/834133/Intel%20AI_LLM%20Model%20Inference%20Using%20IPEX-LLM_Whitepaper_rev1.0.pdf

220. SafeDialBench: A Fine-Grained Safety Evaluation Benchmark for ... https://arxiv.org/html/2502.11090

221. [PDF] Dimensionality, Language, Culture and Safety at DSTC 12 https://aclanthology.org/2025.dstc-1.3.pdf

222. [PDF] Engagingness in Open-Domain Dialogue Systems https://cdnintech.com/articles/840/1776057711-426321750/acrt20250081-r22026-04-09%2007%3A25%3A40.pdf

223. [PDF] Tokenization Is More Than Compression - ACL Anthology https://aclanthology.org/2024.emnlp-main.40.pdf

224. Streaming Technologies and Serialization Protocols - arXiv https://arxiv.org/html/2407.13494v1

225. A Comprehensive Overview of Large Language Models https://dl.acm.org/doi/full/10.1145/3744746

226. Application of retrieval-augmented generation for interactive ... https://www.sciencedirect.com/science/article/pii/S0920548925000248

227. A unified multimodal GenAI platform integrating GraphRAG multi ... https://www.nature.com/articles/s41598-026-47145-x

228. Medical LLMs: Fine-Tuning vs. Retrieval-Augmented Generation https://pmc.ncbi.nlm.nih.gov/articles/PMC12292519/

229. [PDF] The Streaming Batch Model for Efficient and Fault-Tolerant ... - arXiv https://arxiv.org/pdf/2501.12407

230. Large Language Model as Token Compressor and Decompressor https://arxiv.org/html/2603.25340v2

231. What Are AI Tokens? The Language and Currency Powering ... https://blogs.nvidia.com/blog/ai-tokens-explained/

232. [PDF] AI Privacy Risks & Mitigations – Large Language Models (LLMs) https://www.edpb.europa.eu/system/files/2025-04/ai-privacy-risks-and-mitigations-in-llms.pdf

233. Datasets for large language models: a comprehensive survey https://link.springer.com/article/10.1007/s10462-025-11403-7

234. Domain Specific Benchmarks for Evaluating Multimodal ... - arXiv https://arxiv.org/html/2506.12958v2

235. A Comprehensive Overview of Large Language Models https://dl.acm.org/doi/10.1145/3744746

236. Survey of neural network optimization methods for sustainable AI https://www.sciencedirect.com/science/article/pii/S2666827025001458

237. Building Domain-Specific Small Language Models via Guided Data ... https://arxiv.org/html/2511.21748v1

238. LogoXpertNet: a novel lightweight logo classification using deep ... https://www.nature.com/articles/s41598-026-45682-z

239. Explainable Deep Learning Framework for Binary Corrosion Image ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12656319/

240. Efficiency-accuracy trade-offs in fine-grained dog breed classification https://link.springer.com/article/10.1007/s42452-025-07798-1

241. ARTICLE IN PRESS - Nature https://www.nature.com/articles/s41598-026-47145-x_reference.pdf

242. A Comprehensive Survey of Small Language Models in the Era of ... https://dl.acm.org/doi/full/10.1145/3768165

243. [PDF] TabArena: A Living Benchmark for Machine Learning on Tabular Data https://hal.science/hal-05136945v2/file/TabArena_%20A%20Living%20Benchmark%20for%20Machine%20Learning%20on%20Tabular%20Data%20%28NeurIPS%29.pdf

244. [PDF] Dialogue Benchmark Generation from Knowledge Graphs with Cost ... https://arxiv.org/pdf/2501.09928

245. HPC-LLM: Practical Domain Adaptation and Retrieval-Augmented ... https://arxiv.org/html/2605.16347v1

246. A Comprehensive Overview of Large Language Models - arXiv https://arxiv.org/html/2307.06435v9

247. RAGPerf: An End-to-End Benchmarking Framework for Retrieval ... https://arxiv.org/html/2603.10765v1

248. [PDF] Dialogue-Based Data Generation for LLM Code Translation - arXiv https://arxiv.org/pdf/2512.03086

249. [PDF] A Comprehensive Overview of Large Language Models - arXiv https://arxiv.org/pdf/2307.06435

250. OpenCoder: The Open Cookbook for Top-Tier Code Large ... - arXiv https://arxiv.org/html/2411.04905v1

251. Understanding LLMs: A Comprehensive Overview from Training to ... https://arxiv.org/html/2401.02038v2

252. A Hypernetwork-Driven Meta-Gated LLM - arXiv https://arxiv.org/html/2605.01973v1

253. [PDF] Selection of LLM Fine-Tuning Data Based on Orthogonal Rules https://ojs.aaai.org/index.php/AAAI/article/view/40444/44405

254. Improving Data Efficiency via Curating LLM-Driven Rating Systems https://openreview.net/forum?id=DKkQtRMowq

255. [PDF] PINTOK: TOKENIZERS DESERVE DEDICATED PINNED CPU ... https://openreview.net/pdf?id=sGMbneFX2n

256. Cross-Lingual Token Arbitrage: Optimizing Code Agent Context ... https://arxiv.org/html/2606.03618v1

257. Mastering LLM Techniques: Text Data Processing - NVIDIA Developer https://developer.nvidia.com/blog/mastering-llm-techniques-data-preprocessing/

258. (PDF) A Survey of LLM × DATA - ResearchGate https://www.researchgate.net/publication/392105529_A_Survey_of_LLM_times_DATA

259. Markdown is 15% more token efficient than JSON - Prompting https://community.openai.com/t/markdown-is-15-more-token-efficient-than-json/841742

260. Is there any benefit to tokenizing and then building an AST? https://stackoverflow.com/questions/25111057/is-there-any-benefit-to-tokenizing-and-then-building-an-ast

261. Live-Coding: Tokenization - Kaggle https://www.kaggle.com/code/rtatman/rebeccaturner-live-coding-tokenization

262. SAFE: Improving LLM Systems using Sentence-Level In-generation ... https://arxiv.org/html/2505.12621v2

263. System Log Parsing with Large Language Models: A Review - arXiv https://arxiv.org/html/2504.04877v2